todos:
- think about vertex centric algorithms in our setting
- after reading StreamVByte think about faster decoding

Ligra+

Main Ideas to remember: push/pull BFS optimization, edge representation compression

Ligra/Intro

- Builds upon Ligra to give better performance for in memory parallel graph algorithms
- Provides a vertex subset data structure in sparse and dense representations, and implementations of edgemap and vertexmap
  - These primitives are sufficient to implement stuff like BFS
  - Vertexmap takes some possibly side effecting funciton $f : V \to \{T, F\}$ and returns the vertices in the subset that satisfy f
    - So it's just a filter over the vertices I guess? You can't add new vertices? I guess it's not sufficient for the BFS update then?
    - If it is just a filter then you implement it in parallel as you do with filter, first compute a boolean array then scan over these guys then copy
  - EdgeMap has a more complicated signature, you take some VertexMap which is your valid sources and a edge predicate $f : E \to \{T, F\}$ which is possibly side effecting (called the update), as well as a vertex predicate $g : V \to \{T, F\}$ called the condition. It then returns all of the **vertices** for which $f(u, v)$ is true where u is in the old frontier, and also $g(v)$ holds.
    - It's more or less all of the things you need to specify the BFS frontier update (and can work for more general algorithms), in BFS you take f to be "this edge gets included if it wins the compare and swap for visiting v" and you take g to be that the vertex must not be visited
    - Implementation depends on sparse/dense

Sparse and Dense Representations, Push/Pull Optimization

- You can represent the vertex subset either sparsely (using a list of vertices) or densely (using a bitmap)
- The main idea is that depending on how large our frontier is, EdgeMap switches implementations for better efficiency
  - In the case where we have few vertices in our frontier we'll do a normal push style bfs where each vertex expands out
  - In the case where we have many vertices we'll do a pull BFS where we instead look over the other vertices and ask if any will be expanded into by the frontier
  - So in either case we try to avoid having to look over all the edges in the graph, if frontier is sparse then $N(F)$ smaller than m, if frontier is dense then we'll hopefully be able to quickly identify, from some vertex outside the frontier, whether it gets expanded into by querying the bitmap
- Implementation looks like:
  - Sparse vertexSubset:
    - In parallel over the vertices of the frontier, get the neighborhood, call the update function over all the edges and filter, then collect all the resulting lists, I guess we can filter for g at the end (whoops after checking you filter for g up front and f assumes g is satisfied)
  - Dense vertexSubset:
    - In parallel over the vertices satisfying g, query the in neighbors of this vertex (we assume access to these) and for each query in $O(1)$ whether it's in the frontier, if so add ourselves to the next frontier if we satisfy g the condition function

Compression Schemes

- Parallel graph algorithms are actually memory limited (at least they were at the time of writing) and so parallel algorithms can see less speedup as expected
  - If we compress the graph, we can save on storage and fit larger graphs on a non-distributed system
  - The perf hit won't be so bad because we'll be able to read more of the graph with the same fixed bandwidth which hopefully offsets the cost of decoding
- The compression scheme is just to use CSR with a delta encoding, so we sort the adjacency list for a given vertex and store just the deltas, and we use variable int style compression to use less bits as these numbers are $\ll n$ in general so we don't need $\lg n$ bits
  - We then decode on the fly as we process through the graph
- Big note: you'd think that decoding is just a scan but it's a little more complicated because we don't know where one delta begins and ends (var int encoding) so we actually do it sequentially within a "block" and parallelize between blocks with per block offsetes

GraphChi

- We want to do "each vertex has a value" style calculations (so not traversals, more vertex style algs) on a single computer
- Let's offload the graph data to disk and devise a way to lay out for contiguous block reads
- We split $[n]$ into P contiguous parts of such that each shard can fit in memory with roughly the same memory footprint (we want one shard and one window to fit, as defined below, in general vertices follow a power law type degree distribution so this isn't balanced in number of vtx)
- Associate with each part a "shard" which contains all edges (u, v) such that v lives in this part of $[n]$, with shards living on disk
- The shard's data is sorted by the index of u; this means that for a given part we can find all of its edge data if have the current shard in memory (this gives all of the in neighbors) and we sequentially read from the other $P - 1$ shards left to right (reading left to right gives all of the first vertex's neighbors, then the second and so on)
- So assuming our desired algorithm operates with some update function which maps vertices to values, as a function of its in and out neighbors, GraphChi proceeds like:
  - For each of the P parts, process by doing sequential block reads over the other $P - 1$ shards and calling the update function in parallel over vertices whose out neighbors have been read
  - You don't read the whole other part at a time, just read in blocks, update in parallel, and write back
  - you send reads to be completed while the compute is still happening (at least I think so, this is async io)
  - so in general you do $O(P^2)$ IOs which isn't great, I believe you'd expect to be limited by bandwidth here to a far greater extent than in memory algorithms and you're somewhat competitive with distributed systems which have to syncronize after each round
- Okay so the reason this was huge is that we're optimizing for as few random reads as possible and this layout allows us to only do block reads for certain types of graph algorithms
  - so think hard drives, minimize number of seeks, although the paper does mention SSDs